EXPERIMENT NO. 8

<u>AIM:</u> CRUD application using Student dabase in mongo and test cases using mocha framework

<u>THEORY:</u>

Introduction

This experiment demonstrates the development of a full-stack web application implementing basic CRUD (Create, Read, Update, Delete) operations using MongoDB, Express.js, and React.js. The application allows users to manage ward records by adding, viewing, updating, and deleting data through an interactive user interface.

MongoDB is used as a NoSQL database to store ward data, while Express.js and Node.js handle backend operations and API creation. React.js is used to build a dynamic frontend interface that interacts with the backend using HTTP requests.

This modular approach separates concerns between frontend and backend, improving scalability, maintainability, and performance.

System Architecture

The application follows a **client-server architecture**:

- **Frontend (React.js)** – Handles user interface and interactions
- **Backend (Node.js + Express.js)** – Processes requests and business logic
- **Database (MongoDB)** – Stores ward data

The frontend communicates with the backend through RESTful APIs using HTTP methods such as GET, POST, PUT, and DELETE.

MongoDB and Mongoose

State is used to store and manage dynamic data in a React component. In this experiment, the **useState hook** is used to manage values like count and text.

Example:

- Count value changes when Increment/Decrement buttons are clicked
- Conditional messages like "Number is odd" are displayed based on state

State updates automatically re-render the UI, ensuring real-time updates without refreshing the page.

CRUD Operations

Create Operation

Allows users to add a new ward to the database.
Data is sent from the frontend to the backend using a POST request and stored in MongoDB.

Read Operation

Fetches and displays all ward records.
The frontend sends a GET request to retrieve data from the database and renders it dynamically.

Update Operation

Enables modification of existing ward data.
A PUT request is used to update specific records based on their unique ID.

Delete Operation

Removes a ward record from the database.
A DELETE request is sent to the backend to remove the selected entry.

React Components

The frontend application is built using reusable React components:

- **Form Component** – Handles input for adding and updating wards
- **WardList Component** – Displays all ward records
- **WardItem Component** – Represents individual ward entries
- **Button Component** – Used for actions like add, update, delete

These components improve code reusability and modularity.

Mocha

Mocha is a JavaScript testing framework used to test Node.js applications. It helps verify whether the backend APIs are working correctly by running test cases and checking expected outputs.

In this experiment, Mocha is used to test CRUD operations implemented in the backend. It ensures that API endpoints return correct responses and handle data properly.

CODE:

App.js

```
import { useState } from 'react';
import './App.css';

function App() {
  const [formData, setFormData] = useState({
    name: '',
    prn: '',
    subject: '',
  });
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [isLoadingRecords, setIsLoadingRecords] = useState(false);
  const [message, setMessage] = useState('');
  const [error, setError] = useState('');
  const [submittedData, setSubmittedData] = useState(null);
  const [wardRecords, setWardRecords] = useState([]);
  const [editingWardId, setEditingWardId] = useState(null);
  const [editingWardData, setEditingWardData] = useState({
    name: '',
    prn: '',
    subject: '',
  });

  const handleChange = (event) => {
    const { name, value } = event.target;

    setFormData((currentData) => ({
      ...currentData,
      [name]: value,
    }));
  };

  const handleSubmit = async (event) => {
    event.preventDefault();
    setIsSubmitting(true);
    setMessage('');
    setError('');

    try {
      const response = await fetch('http://localhost:5000/api/wards', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
```

```
      },
      body: JSON.stringify(formData),
    });

    if (!response.ok) {
      const errorData = await response.json().catch(() => ({}));
      throw new Error(errorData.message || 'Failed to submit ward details');
    }

    const savedWard = await response.json();
    setSubmittedData(savedWard);
    setMessage('Ward details saved successfully.');
    setFormData({
      name: '',
      prn: '',
      subject: '',
    });
  } catch (submitError) {
    setError(submitError.message);
  } finally {
    setIsSubmitting(false);
  }
};

const loadWardRecords = async () => {
  setIsLoadingRecords(true);
  setMessage('');
  setError('');

  try {
    const response = await fetch('http://localhost:5000/api/wards');

    if (!response.ok) {
      throw new Error('Failed to load ward records');
    }

    const records = await response.json();
    setWardRecords(records);
    setMessage(records.length ? 'Ward records loaded.' : 'No ward records found.');
  } catch (loadError) {
    setError(loadError.message);
  } finally {
    setIsLoadingRecords(false);
  }
};
```

```
const handleEditClick = (record) => {
  setEditingWardId(record._id);
  setEditingWardData({
    name: record.name,
    prn: record.prn,
    subject: record.subject,
  });
  setMessage('');
  setError('');
};

const handleEditingWardChange = (event) => {
  const { name, value } = event.target;

  setEditingWardData((currentData) => ({
    ...currentData,
    [name]: value,
  }));
};

const handleCancelEdit = () => {
  setEditingWardId(null);
  setEditingWardData({
    name: '',
    prn: '',
    subject: '',
  });
};

const handleSaveEdit = async (event) => {
  event.preventDefault();

  if (!editingWardId) {
    return;
  }

  setIsSubmitting(true);
  setMessage('');
  setError('');

  try {
    const response = await fetch(`http://localhost:5000/api/wards/${editingWardId}`, {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json',
      },
```

```
      body: JSON.stringify(editingWardData),
    });

    if (!response.ok) {
      const errorData = await response.json().catch(() => ({}));
      throw new Error(errorData.message || 'Failed to update ward details');
    }

    const updatedWard = await response.json();

    setWardRecords((currentRecords) =>
      currentRecords.map((record) => (record._id === updatedWard._id ? updatedWard :
record))
    );
    setSubmittedData(updatedWard);
    setMessage('Ward details updated successfully.');
    handleCancelEdit();
  } catch (saveError) {
    setError(saveError.message);
  } finally {
    setIsSubmitting(false);
  }
};

const handleDeleteWard = async (wardId) => {
  setIsSubmitting(true);
  setMessage('');
  setError('');

  try {
    const response = await fetch(`http://localhost:5000/api/wards/${wardId}`, {
      method: 'DELETE',
    });

    if (!response.ok) {
      const errorData = await response.json().catch(() => ({}));
      throw new Error(errorData.message || 'Failed to delete ward');
    }

    setWardRecords((currentRecords) =>
      currentRecords.filter((record) => record._id !== wardId)
    );

    if (editingWardId === wardId) {
      handleCancelEdit();
    }
```

```
    setMessage('Ward deleted successfully.');
  } catch (deleteError) {
    setError(deleteError.message);
  } finally {
    setIsSubmitting(false);
  }
};

return (
  <main className="app-shell">
    <section className="form-card">
      <p className="eyebrow">Student Registration</p>
      <h1>Submit your details</h1>
      <p className="subtitle">Fill in your name, PRN, and subject to register the entry.</p>

      <form className="student-form" onSubmit={handleSubmit}>
        <label>
          <span>Name</span>
          <input
            type="text"
            name="name"
            value={formData.name}
            onChange={handleChange}
            placeholder="Enter your name"
            required
          />
        </label>

        <label>
          <span>PRN</span>
          <input
            type="text"
            name="prn"
            value={formData.prn}
            onChange={handleChange}
            placeholder="Enter your PRN"
            required
          />
        </label>

        <label>
          <span>Subject</span>
          <input
            type="text"
            name="subject"
```

```
          value={formData.subject}
          onChange={handleChange}
          placeholder="Enter your subject"
          required
        />
      </label>

      <button type="submit">Submit</button>
    </form>

    <div className="actions-row">
      <button type="button" className="secondary-button" onClick={loadWardRecords}>
        {isLoadingRecords ? 'Loading...' : 'Load Ward Records'}
      </button>
    </div>

    {(message || error) && (
      <div className={`status-banner ${error ? 'status-error' : 'status-success'}`}>
        {error || message}
      </div>
    )}

    {submittedData && (
      <div className="result-panel">
        <h2>Submitted Details</h2>
        <p><strong>Name:</strong> {submittedData.name}</p>
        <p><strong>PRN:</strong> {submittedData.prn}</p>
        <p><strong>Subject:</strong> {submittedData.subject}</p>
      </div>
    )}

    <div className="records-panel">
      <h2>Current Wards</h2>
      {wardRecords.length === 0 ? (
        <p className="empty-state">Click Load Ward Records to view the current
entries.</p>
      ) : (
        <div className="records-list">
          {wardRecords.map((record) => (
            <article className="record-item" key={record._id}>
              {editingWardId === record._id ? (
                <form className="record-edit-form" onSubmit={handleSaveEdit}>
                  <div className="record-header">
                    <h3>Edit Ward</h3>
                    <button
                      type="button"
```

```
    className="icon-button close-button"
    onClick={handleCancelEdit}
    aria-label="Cancel edit"
  >
    ×
  </button>
</div>

<label>
  <span>Name</span>
  <input
    type="text"
    name="name"
    value={editingWardData.name}
    onChange={handleEditingWardChange}
    required
  />
</label>

<label>
  <span>PRN</span>
  <input
    type="text"
    name="prn"
    value={editingWardData.prn}
    onChange={handleEditingWardChange}
    required
  />
</label>

<label>
  <span>Subject</span>
  <input
    type="text"
    name="subject"
    value={editingWardData.subject}
    onChange={handleEditingWardChange}
    required
  />
</label>

<div className="record-actions">
  <button type="submit" className="save-button" disabled={isSubmitting}>
    {isSubmitting ? 'Saving...' : 'Save'}
  </button>
```

```
                <button type="button" className="cancel-button"
onClick={handleCancelEdit}>
                  Cancel
                </button>
              </div>
            </form>
          ) : (
            <>
              <div className="record-header">
                <h3>{record.name}</h3>
                <div className="record-header-actions">
                  <button
                    type="button"
                    className="icon-button edit-button"
                    onClick={() => handleEditClick(record)}
                    aria-label={`Edit ward ${record.name}`}
                  >
                    ✏️
                  </button>
                  <button
                    type="button"
                    className="icon-button delete-button"
                    onClick={() => handleDeleteWard(record._id)}
                    aria-label={`Delete ward ${record.name}`}
                    disabled={isSubmitting}
                  >
                    🗑️
                  </button>
                </div>
              </div>
              <p><strong>PRN:</strong> {record.prn}</p>
              <p><strong>Subject:</strong> {record.subject}</p>
            </>
          )}
        </article>
      ))}
    </div>
  )}
 </div>
 </section>
 </main>
 );
}

export default App;
```

index.js

```
const express = require('express')
const app = express()
const mongoose = require('mongoose');
const cors = require('cors');
const path = require("path");
const Ward = require('./models/ward');
require("dotenv").config({
  path: path.join(__dirname, ".env")
});

const PORT = process.env.PORT || 5000;

// Middleware
app.use(express.json());
app.use(cors());

const MONGO_URI = process.env.MONGO_URI;

async function connectToDatabase() {
  try {
    await mongoose.connect(MONGO_URI);
    console.log(" Connected to MongoDB");
  } catch (err) {
    console.error(" MongoDB connection error:", err);
  }
}

app.get('/', (req, res) => {
  res.send('Hello World!')
})

app.get('/api/wards', async (req, res) => {
  try {
    const wards = await Ward.find().sort({ createdAt: -1 });
    res.json(wards);
  } catch (error) {
    res.status(500).json({ message: 'Failed to load wards' });
  }
});

app.post('/api/wards', async (req, res) => {
  try {
    const { name, prn, subject } = req.body;
```

```
    if (!name || !prn || !subject) {
      return res.status(400).json({ message: 'Name, PRN, and subject are required' });
    }

    const ward = await Ward.create({ name, prn, subject });
    res.status(201).json(ward);
  } catch (error) {
    res.status(500).json({ message: 'Failed to save ward' });
  }
});

app.put('/api/wards/:id', async (req, res) => {
  try {
    const { id } = req.params;
    const { name, prn, subject } = req.body;

    if (!name || !prn || !subject) {
      return res.status(400).json({ message: 'Name, PRN, and subject are required' });
    }

    const updatedWard = await Ward.findByIdAndUpdate(
      id,
      { name, prn, subject },
      { new: true, runValidators: true }
    );

    if (!updatedWard) {
      return res.status(404).json({ message: 'Ward not found' });
    }

    res.json(updatedWard);
  } catch (error) {
    res.status(500).json({ message: 'Failed to update ward' });
  }
});

app.delete('/api/wards/:id', async (req, res) => {
  try {
    const { id } = req.params;
    const deletedWard = await Ward.findByIdAndDelete(id);

    if (!deletedWard) {
      return res.status(404).json({ message: 'Ward not found' });
    }
```

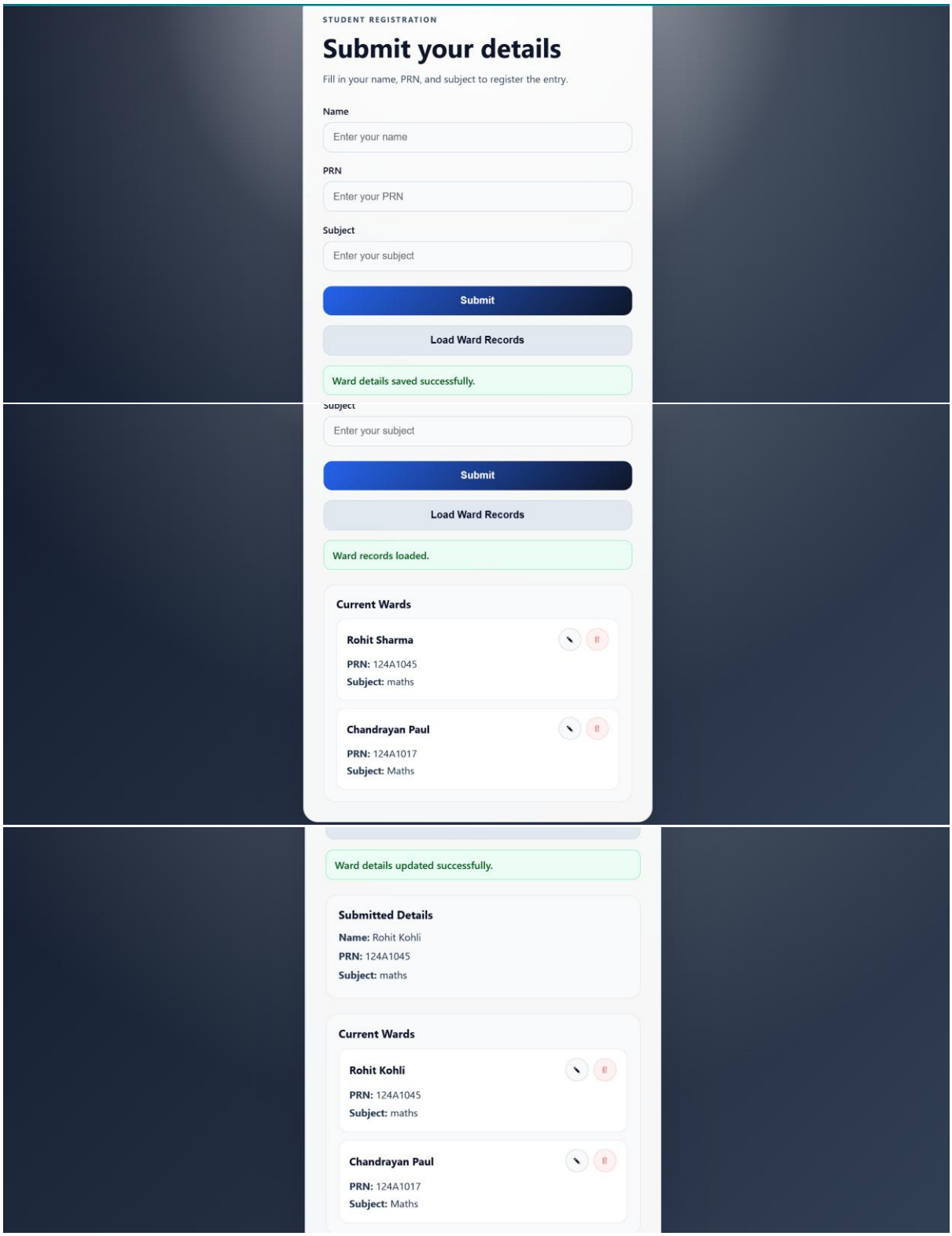
```
    res.json({ message: 'Ward deleted successfully' });
  } catch (error) {
    res.status(500).json({ message: 'Failed to delete ward' });
  }
});

async function startServer() {
  await connectToDatabase();
  app.listen(PORT, () => {
    console.log(`Example app listening on port ${PORT}`);
  });
}

if (require.main === module) {
  startServer();
}

module.exports = {
  app,
  connectToDatabase,
  startServer,
};
```

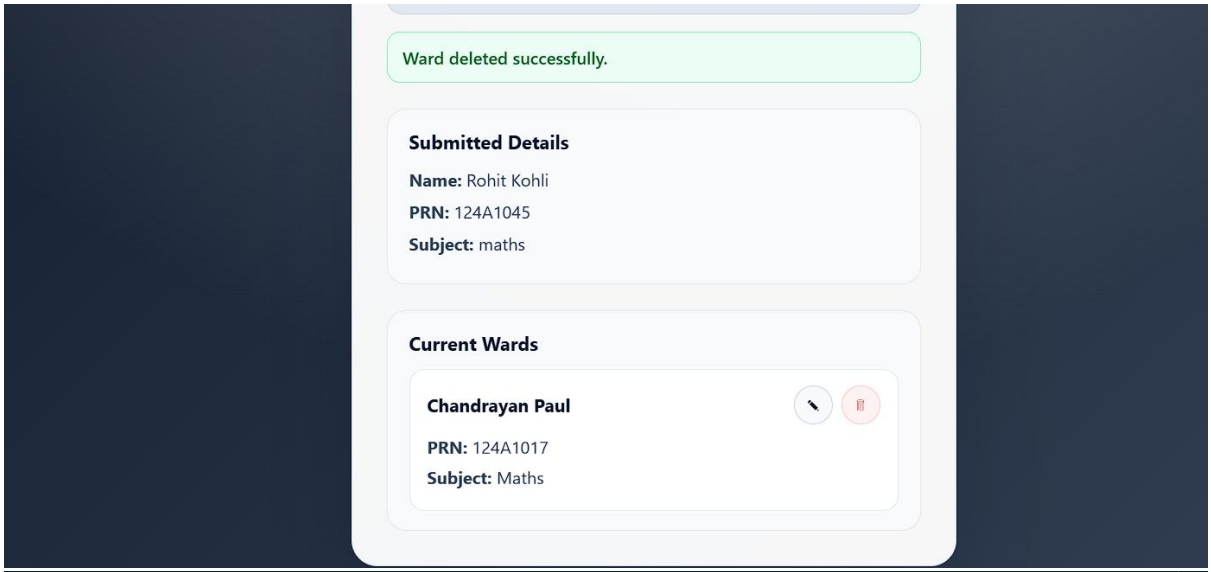
ward.js

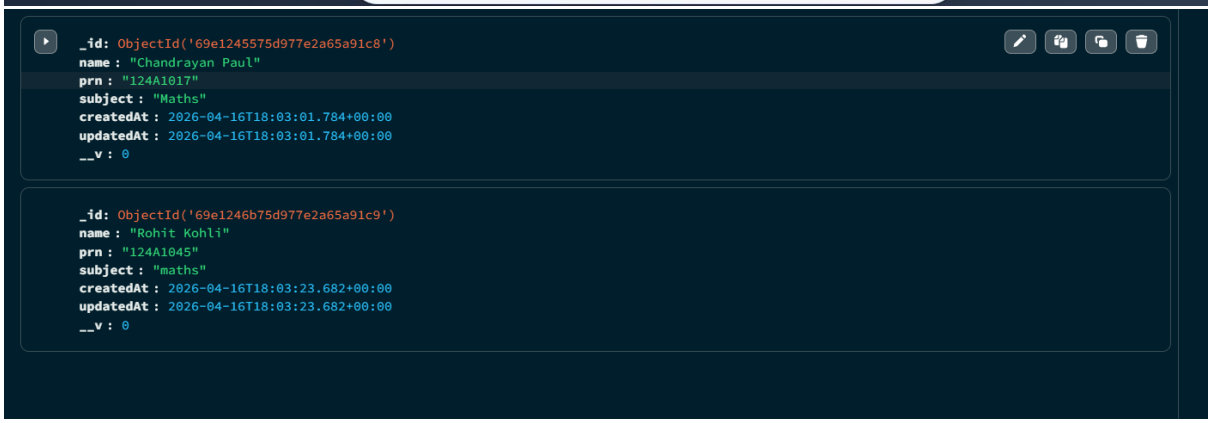
```
const mongoose = require('mongoose');

const wardSchema = new mongoose.Schema(
    {
        name: {
            type: String,
            required: true,
            trim: true,
        },
        prn: {
            type: String,
            required: true,
            trim: true,
        },
        subject: {
            type: String,
            required: true,
            trim: true,
        },
```

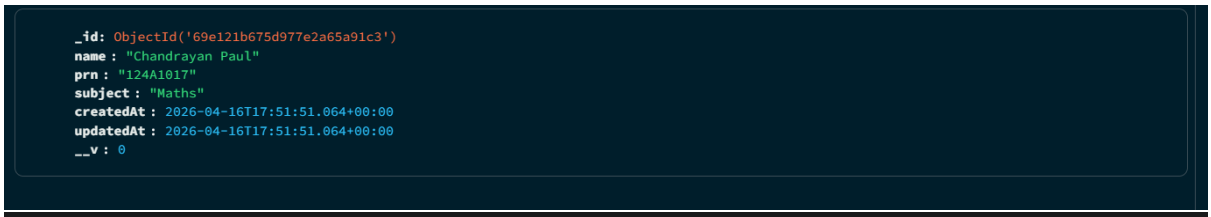
```
  },
  {
    timestamps: true,
  }
);

module.exports = mongoose.model('Ward', wardSchema);
```

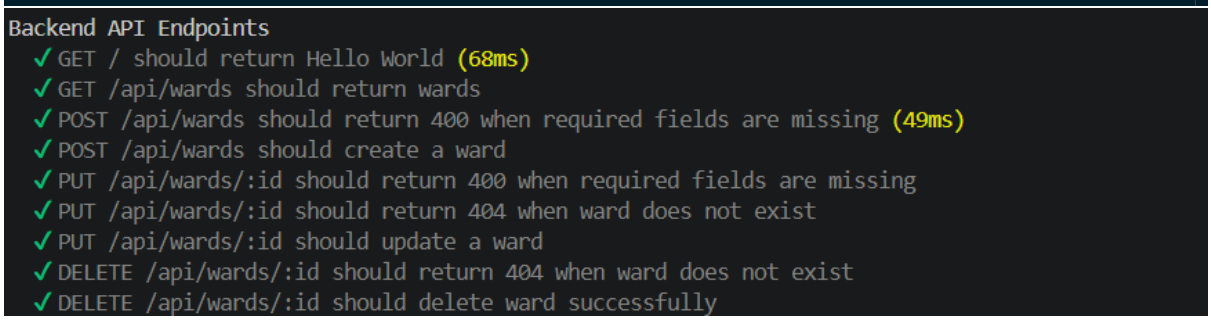
OUTPUT:











CONCLUSION: This experiment successfully demonstrates the implementation of a full-stack CRUD application using MongoDB, Express.js, and React.js. It highlights the importance of component-based architecture, state management, and API integration in modern web development.
The system can be further enhanced by adding authentication, validation, and advanced features like search and filtering.